

RESTful Schema-Aware Data Integration and Transformation API

Dijith Dinesh

Aaron Paul

Peter Joe Menachery

Athuljith S

Pranav Manoj

Department of Computer Science and Engineering
Model Engineering College, Thrikkakara, Kochi 682021

August 07, 2025

Table of Contents

- 1 Introduction
- 2 Problem Statement
- 3 Objectives
- 4 Proposed Solution
- 5 Existing Solution Limitations
- 6 Example API Calls
- 7 Block Diagram
- 8 Technology Stack
- 9 Modules and Duties
- 10 Expected Outcome
- 11 Challenges and Risks
- 12 Conclusion
- 13 References

- This project proposes a universal API that is schema-aware to serve as an intelligent data gateway.

- This project proposes a universal API that is schema-aware to serve as an intelligent data gateway.
- Modern applications suffer from brittle integrations, where small backend changes cause major client-side failures.

- This project proposes a universal API that is schema-aware to serve as an intelligent data gateway.
- Modern applications suffer from brittle integrations, where small backend changes cause major client-side failures.
- Our system aims to solve the core problems of **tight coupling**, **data transformation overhead**, and **insecure data access** through an object-oriented design.

Problem Statement

- **Tight Coupling:** Client applications are directly tied to the database schema. A simple column rename in the database can break multiple applications.

Problem Statement

- **Tight Coupling:** Client applications are directly tied to the database schema. A simple column rename in the database can break multiple applications.
- **Developer Inefficiency:** Engineers waste significant time writing repetitive boilerplate code to fetch, parse, and re-format data for different use cases.

Problem Statement

- **Tight Coupling:** Client applications are directly tied to the database schema. A simple column rename in the database can break multiple applications.
- **Developer Inefficiency:** Engineers waste significant time writing repetitive boilerplate code to fetch, parse, and re-format data for different use cases.
- **Security Risks:** Direct database exposure or poorly managed APIs create vulnerabilities and make it difficult to enforce consistent access policies.

Objectives

- To design a RESTful API that serves as an **abstraction layer** over a database, decoupling clients from the schema.

Objectives

- To design a RESTful API that serves as an **abstraction layer** over a database, decoupling clients from the schema.
- To build a **dynamic query engine** that translates simple URL parameters into secure SQL queries.

Objectives

- To design a RESTful API that serves as an **abstraction layer** over a database, decoupling clients from the schema.
- To build a **dynamic query engine** that translates simple URL parameters into secure SQL queries.
- To implement **on-the-fly data transformation** into multiple formats (JSON, CSV, XML) with dynamic field renaming.

Objectives

- To design a RESTful API that serves as an **abstraction layer** over a database, decoupling clients from the schema.
- To build a **dynamic query engine** that translates simple URL parameters into secure SQL queries.
- To implement **on-the-fly data transformation** into multiple formats (JSON, CSV, XML) with dynamic field renaming.
- To enforce **role-based access control (RBAC)** using API keys for enhanced security.

Objectives

- To design a RESTful API that serves as an **abstraction layer** over a database, decoupling clients from the schema.
- To build a **dynamic query engine** that translates simple URL parameters into secure SQL queries.
- To implement **on-the-fly data transformation** into multiple formats (JSON, CSV, XML) with dynamic field renaming.
- To enforce **role-based access control (RBAC)** using API keys for enhanced security.
- To demonstrate core **Object-Oriented Programming principles** like Inheritance, Polymorphism, and Encapsulation.

- **A Decoupled API Abstraction Layer:**
 - A lightweight, RESTful middleware that intelligently translates simple client requests into complex, secure database queries.

- **A Decoupled API Abstraction Layer:**
 - A lightweight, RESTful middleware that intelligently translates simple client requests into complex, secure database queries.
- **Core Features:**
 - **Dynamic Querying:** Allows filtering and sorting data via simple URL parameters.
 - **On-the-Fly Transformation:** Lets clients request data in any format and rename fields as needed.

- **A Decoupled API Abstraction Layer:**

- A lightweight, RESTful middleware that intelligently translates simple client requests into complex, secure database queries.

- **Core Features:**

- **Dynamic Querying:** Allows filtering and sorting data via simple URL parameters.
- **On-the-Fly Transformation:** Lets clients request data in any format and rename fields as needed.
- **Centralized Security:** Manages access through role-based API keys.
 - We intentionally use only API keys for authentication. This gives developers the freedom to integrate any user authentication system (e.g., OAuth, JWT) on their end, as our API only needs the final, secure key.

Existing Solution Limitations

- **Direct Database Access:** Insecure, brittle, and not scalable.

Existing Solution Limitations

- **Direct Database Access:** Insecure, brittle, and not scalable.
- **Backend-as-a-Service (BaaS):** Offers limited transformation flexibility and can lead to vendor lock-in.

Existing Solution Limitations

- **Direct Database Access:** Insecure, brittle, and not scalable.
- **Backend-as-a-Service (BaaS):** Offers limited transformation flexibility and can lead to vendor lock-in.
- **Manual API Development:** High effort, error-prone, and leads to inconsistent, repetitive codebases.

Existing Solution Limitations

- **Direct Database Access:** Insecure, brittle, and not scalable.
- **Backend-as-a-Service (BaaS):** Offers limited transformation flexibility and can lead to vendor lock-in.
- **Manual API Development:** High effort, error-prone, and leads to inconsistent, repetitive codebases.
- **Enterprise API Gateways:** Powerful but often too complex and expensive for many projects.

Example API Calls

Example 1: Complex Query with JSON Output

Request: /data/phones?where=price:gt:50000&sort=price:desc&format=json&replace=id:no

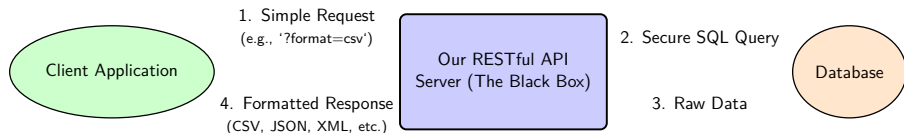
Explanation: Fetches products costing more than 50000, sorted by price, returning the items with the 'id' field renamed to 'no' in JSON format.

Actual Data: id:number,name:string,price:number

Output:

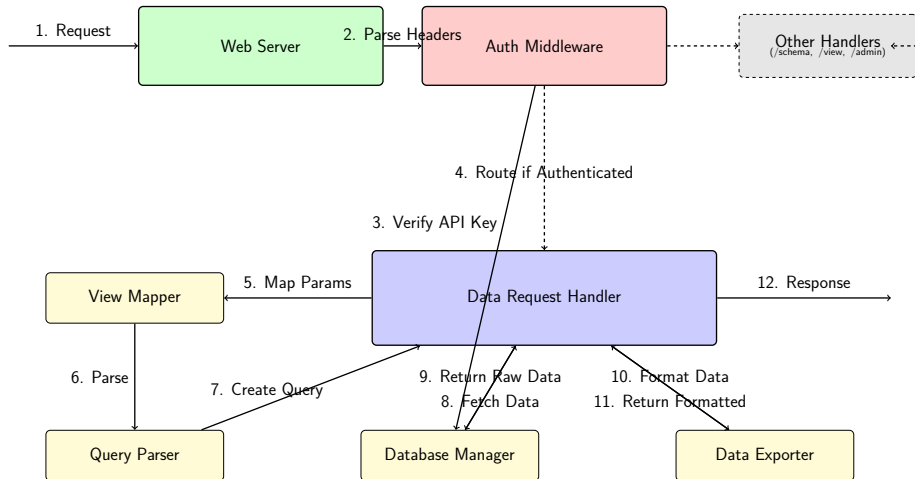
```
[
  { "no": 101, "name": "pixel","price":79999 },
  { "no": 205, "name": "one plus","price":51999 }
]
```

Block Diagram: High-Level Architecture



- Our API acts as an intelligent intermediary, abstracting away database complexity.

Block Diagram: Internal Request Flow



- **Programming Language:**
 - Java (JDK 11+)

Technology Stack

- **Programming Language:**
 - Java (JDK 11+)
- **Frameworks / Libraries:**
 - **Core:** Standard Java Libraries (no external frameworks)
 - **Networking:** 'com.sun.net.httpserver'
 - **Database:** JDBC API
 - **JSON Processing:** Gson

Technology Stack

- **Programming Language:**

- Java (JDK 11+)

- **Frameworks / Libraries:**

- **Core:** Standard Java Libraries (no external frameworks)
- **Networking:** 'com.sun.net.httpserver'
- **Database:** JDBC API
- **JSON Processing:** Gson

- **Tools:**

- **Database:** SQLite (initially for simplicity).
 - Our design encapsulates all DB logic, allowing a seamless switch to a more powerful system like PostgreSQL without any impact on the end-user.
- **IDE:** VS Code, Neovim (with JDTLS LSP)
- **Version Control:** Git

- **API Server Core:** Starts the server and routes incoming requests to appropriate handlers.

- **API Server Core:** Starts the server and routes incoming requests to appropriate handlers.
- **Request Handling:** Parses HTTP requests. A base handler provides common logic, while concrete handlers manage specific endpoints ('/data', '/schema').

- **API Server Core:** Starts the server and routes incoming requests to appropriate handlers.
- **Request Handling:** Parses HTTP requests. A base handler provides common logic, while concrete handlers manage specific endpoints ('/data', '/schema').
- **Query Parsing Engine:** Translates user-friendly URL parameters (e.g., 'where=...') into secure, parameterized SQL.

- **API Server Core:** Starts the server and routes incoming requests to appropriate handlers.
- **Request Handling:** Parses HTTP requests. A base handler provides common logic, while concrete handlers manage specific endpoints ('/data', '/schema').
- **Query Parsing Engine:** Translates user-friendly URL parameters (e.g., 'where=...') into secure, parameterized SQL.
- **Database Interaction Layer:** Encapsulates all JDBC logic, providing a clean interface for data operations.

- **API Server Core:** Starts the server and routes incoming requests to appropriate handlers.
- **Request Handling:** Parses HTTP requests. A base handler provides common logic, while concrete handlers manage specific endpoints ('/data', '/schema').
- **Query Parsing Engine:** Translates user-friendly URL parameters (e.g., 'where=...') into secure, parameterized SQL.
- **Database Interaction Layer:** Encapsulates all JDBC logic, providing a clean interface for data operations.
- **Data Exporter:** Polymorphically converts Java objects into client-requested formats like JSON, CSV, or XML.

Expected Outcome

- **Final Deliverable:** A fully functional, standalone RESTful API server packaged as a runnable JAR file.

Expected Outcome

- **Final Deliverable:** A fully functional, standalone RESTful API server packaged as a runnable JAR file.
- **Measurable Result:** A working prototype that successfully decouples client requests from the backend database schema.

Expected Outcome

- **Final Deliverable:** A fully functional, standalone RESTful API server packaged as a runnable JAR file.
- **Measurable Result:** A working prototype that successfully decouples client requests from the backend database schema.
- The prototype will demonstrate all core features:
 - Dynamic querying and data transformation.
 - Multi-format data export (JSON, CSV, XML, etc.).
 - Secure, role-based access control.

Expected Outcome

- **Final Deliverable:** A fully functional, standalone RESTful API server packaged as a runnable JAR file.
- **Measurable Result:** A working prototype that successfully decouples client requests from the backend database schema.
- The prototype will demonstrate all core features:
 - Dynamic querying and data transformation.
 - Multi-format data export (JSON, CSV, XML, etc.).
 - Secure, role-based access control.
- Well-documented source code exemplifying Object-Oriented design patterns.

Challenges

- Designing a robust and secure query parser to prevent SQL injection.

Challenges

- Designing a robust and secure query parser to prevent SQL injection.
- Ensuring the polymorphic exporter design is flexible enough for different data formats.

Challenges

- Designing a robust and secure query parser to prevent SQL injection.
- Ensuring the polymorphic exporter design is flexible enough for different data formats.
- Managing complex state and logic without relying on external frameworks.

Contingency Plans

- Strictly use 'PreparedStatement' with parameter binding for all database queries.

Challenges

- Designing a robust and secure query parser to prevent SQL injection.
- Ensuring the polymorphic exporter design is flexible enough for different data formats.
- Managing complex state and logic without relying on external frameworks.

Contingency Plans

- Strictly use 'PreparedStatement' with parameter binding for all database queries.
- Adhere to a common 'Exporter' interface and use the Strategy design pattern.

Challenges

- Designing a robust and secure query parser to prevent SQL injection.
- Ensuring the polymorphic exporter design is flexible enough for different data formats.
- Managing complex state and logic without relying on external frameworks.

Contingency Plans

- Strictly use 'PreparedStatement' with parameter binding for all database queries.
- Adhere to a common 'Exporter' interface and use the Strategy design pattern.
- Focus on strong OOP principles (SOLID) to keep the codebase modular and maintainable.

Conclusion

- We propose a system that solves critical data integration challenges through an elegant, object-oriented abstraction layer.

Conclusion

- We propose a system that solves critical data integration challenges through an elegant, object-oriented abstraction layer.
- Our design promotes code reusability and maintainability by applying core OOP principles like Inheritance and Polymorphism.

Conclusion

- We propose a system that solves critical data integration challenges through an elegant, object-oriented abstraction layer.
- Our design promotes code reusability and maintainability by applying core OOP principles like Inheritance and Polymorphism.
- This approach significantly reduces client-side development effort and enhances system security.

Conclusion

- We propose a system that solves critical data integration challenges through an elegant, object-oriented abstraction layer.
- Our design promotes code reusability and maintainability by applying core OOP principles like Inheritance and Polymorphism.
- This approach significantly reduces client-side development effort and enhances system security.
- The final product will be a lightweight, powerful, and flexible tool for any developer needing to expose data securely and efficiently.

Conclusion

- We propose a system that solves critical data integration challenges through an elegant, object-oriented abstraction layer.
- Our design promotes code reusability and maintainability by applying core OOP principles like Inheritance and Polymorphism.
- This approach significantly reduces client-side development effort and enhances system security.
- The final product will be a lightweight, powerful, and flexible tool for any developer needing to expose data securely and efficiently.
- The modular design allows for easy future enhancements, such as implementing HATEOAS to make the API fully self-discoverable.

References



R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” *Ph.D. dissertation, University of California, Irvine*, 2000.

(Fundamental thesis defining REST principles).



J. Bloch, *Effective Java, 3rd Edition*.

Addison-Wesley Professional, 2018.

(Provides best practices for high-quality, object-oriented Java code).



C. Pautasso, O. Zimmermann, and F. Leymann, “RESTful Web Services: Principles, Patterns, Emerging Technologies,” *In: The RESTful Web Services Cookbook: Solutions for Creating, Consuming, and Optimizing Large-Scale Web Services*, 2010.